FEU INSTITUTE OF TECHNOLOGY

COLLEGE OF COMPUTER STUDIES AND MULTIMEDIA ARTS

CS0065
(INTELLIGENT SYSTEMS)

TECHNICAL ASSESSMENT

3

CASE-BASED REASONING (CBR) SYSTEM DEVELOPMENT

Student Name	Paolo C. Gorospe Andreu Liam C. Gamboa
Section:	AN43
Professor:	Crisola G. Tan
Date:	11/01/2026

I. BACKGROUND INFORMATION

Case-Based Reasoning (CBR) is an intelligent problem-solving approach used in **Knowledge Representation and Reasoning (KRR)**. Instead of relying solely on predefined rules, CBR systems solve new problems by referring to past experiences, also known as cases. These cases contain previous problems and their successful solutions. When a new situation arises, the system identifies the most similar past case, adapts the old solution to fit the new context, and applies it. Afterward, the new case is saved, allowing the system to learn and continuously improve.

CBR is widely used in various fields, such as disaster response, legal assistance, and intelligent tutoring, because it mimics how humans naturally make decisions. For example, emergency responders rely on past typhoon data to plan evacuations; lawyers examine previous legal cases to build arguments; and tutoring systems look at prior student mistakes to provide customized guidance. In all these examples, past knowledge becomes the foundation for solving future problems more effectively.

This activity allows students to experience how CBR works by developing a simple, functional CBR program based on real-world scenarios. Through pair programming, students will learn how to represent cases, implement similarity assessment, adapt solutions, and store new learning experiences—reflecting the complete CBR cycle. This hands-on approach strengthens understanding of how intelligent systems use experiential knowledge to support reasoning, decision-making, and learning.

II. TECHNICAL ASSESSMENT

CASE-BASED REASONING (CBR) SYSTEM DEVELOPMENT

Objective

At the end of this laboratory activity, students are expected to:

1. **Understand and apply the Case-Based Reasoning (CBR) cycle**, including retrieval, reuse, revision, and retention.
2. **Represent cases effectively** using appropriate data structures such as dictionaries, arrays, JSON objects, or classes.
3. **Implement a similarity assessment method** to identify relevant past cases based on defined attributes.
4. **Adapt retrieved solutions** to meet the needs of a new problem scenario based on the chosen case study.
5. **Demonstrate learning in CBR** by saving the newly solved case into the system's case base.
6. **Develop collaboration skills** through pair programming and joint problem analysis.

7. **Apply knowledge representation and reasoning concepts** to real-world problems in disaster response, legal reasoning, or intelligent tutoring.

Materials Needed

1. Hardware
   - A laptop or desktop computer
   - Stable internet connection (for research, documentation, submission)

2. Software

A programming environment:
   - Python (recommended: IDLE, VS Code, Jupyter Notebook, PyCharm)
   - Any text editor (Notepad++, Sublime, VS Code) if needed

3. File Formats & Tools

Word processor for documentation:
   - MS Word, Google Docs, or LibreOffice
   - Screenshot tool (e.g., Snipping Tool, Lightshot, Print Screen)

4. Reference Materials

Lecture notes on:
   - Knowledge Representation & Reasoning
   - Case-Based Reasoning cycle
   - Similarity measures

Provided case studies (Disaster DSS, Legal DSS, Intelligent Tutoring System)

5. Pair Collaboration Tools (optional but useful)
   - Google Drive for shared documentation
   - GitHub/GitLab for code collaboration

General Instructions

1. This **Technical Assessment (TA3)** must be completed by Pair.
2. Please ensure that you are grouped as pairs in the People tab.
3. Select only one case study from the list below.
4. Include clear comments in your code to describe its logic.
5. Each program should display the correct input, process, and output.
6. Ensure that your script displays clear and readable outputs.
7. Compile all Python codes and corresponding outputs into one PDF file.
8. Save it as **TA3_lastname1_lastname2.pdf**
   (Example: TA2_Patilano_Loverez.pdf)
9. Submit the **python file** and **PDF** file.

📌 Step-by-Step Instructions

1. Form a pair

Choose a partner. Each pair will work collaboratively on research, coding, debugging, and presenting your results.

2. Choose ONE (1) case study

Select only one case study from the list below. Your program must be based on the scenario and logic of the chosen system:

Case Study Options:

1. *Disaster Response Decision Support System (DSS)*
2. *AI-Based Legal Decision Support System*
3. *Intelligent Tutoring System (ITS)*

3. Define your case base

Create at least **3–5 past cases** stored as dictionaries, JSON objects, arrays, or any structure.

Example structure:

```
{
  "problem": {...},
  "solution": {...}
}
```

4. Implement similarity assessment

Your program must include a way to measure similarity between the new case and past cases.

Examples:

- Attribute matching
- Weighted scoring
- Simple rule-based comparison

Output should show: (sample)

✔ Similarity score or ranking

✔ The most relevant past case retrieved

5. Adapt the solution

Modify the retrieved solution so it fits the new problem scenario.

This part should reflect the logic of your chosen case study:

Examples:

- **DSS:** Adjust evacuation plans based on population size
- **Legal DSS:** Modify arguments due to new digital evidence
- **ITS:** Give personalized feedback based on student errors

6. Demonstrate learning (retain stage)

After generating a final solution, store the new case into your case base.

Your program must show:

✔ “New case saved” or similar confirmation

✔ Updated case list

7. Display clear program outputs

Your output must include:

- New problem description
- Retrieved similar case
- Adapted final solution
- Updated case base

8. Submission Requirements

Submit the following:

A. Program File

- Python

B. Short Documentation (1–2 pages)

Include:

- Title of chosen case study
- Description of your case base
- Explanation of similarity method
- Adaptation logic
- Screenshots of program execution

C. Pair Reflection (5–7 sentences)

Discuss:

- What each partner contributed
- What challenges you encountered
- What you learned about CBR

Case Studies:

Case Study 1: Disaster Response Decision Support System

A CBR-based DSS helps local government units respond to natural disasters.

- Past cases include typhoons, floods, and earthquakes.
- New event: *Typhoon with expected flooding in urban areas.*
- System retrieves similar events: *Typhoon Ursula (2019)* and *Typhoon Odette (2021)*.
- It adapts recommended responses such as evacuation strategy, resource allocation, and shelter capacity.
- After execution, the system stores new outcomes for improved future reasoning.

Learning Outcome:

CBR helps governments respond faster by using *experiential knowledge* rather than rebuilding decisions from scratch.

Case Study 2: AI-Based Legal Decision Support System

A legal CBR tool assists lawyers by retrieving previous court cases similar to the current legal issue.

Scenario:

Client is charged with “theft involving electronic transfer.” System retrieves past cases involving:

- online theft
- digital fraud

- unauthorized bank transfers

Adaptation:
The reasoning engine adjusts argument patterns to match the new technical context.
Learning:
After final verdict, the case is stored—enhancing the system's knowledge for future cybercrime-related offenses.

Case Study 3: Intelligent Tutoring System
A CBR system identifies the learning struggles of students. Student submits a math solution with repeated errors. System retrieves past students with similar error patterns (e.g., misusing formulas). It adapts feedback and personalized lessons based on successful interventions. **Learning:** System improves by retaining new student profiles and outcomes.

Your Code - Laboratory Tasks ⇩

1. Chosen Case Study & System Overview

- **Selected Case Study:** Case Study 3: Intelligent Tutoring System (ITS)
- **Purpose and Scope:** This program recommends tailored interventions for recurring student errors in linear equations. It takes a user-identified error, retrieves a similar historical intervention using weighted attributes, adapts the recommended practice workload, and stores the newly completed case and outcome in persistent storage.
- **Case Representation:** Past cases are represented as JSON/dictionary objects containing:
  - id: Unique identifier (e.g., C001)
  - problem: Error type (sign, distribution, or division), quiz score ($0–100$), and previous attempts ($1–5$)
  - solution: Conceptual feedback, worked step-by-step example, and assigned practice count
  - outcome: Post-intervention score and status (successful, needs_review, or pending)

2. Similarity Assessment & Adaptation Logic

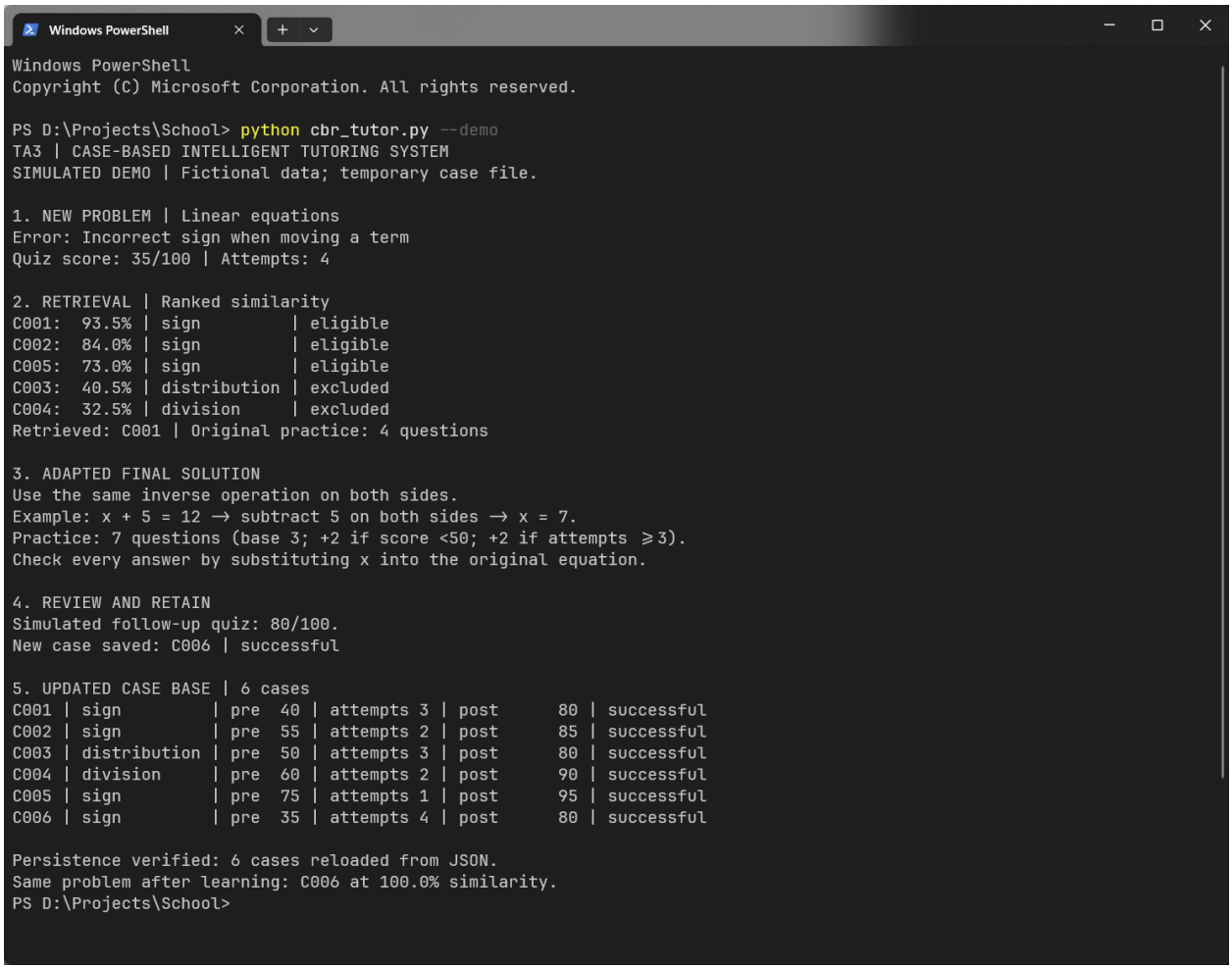
- **Similarity Assessment:** Uses weighted multi-attribute matching where error match carries the highest priority:

$$Similarity = 0.50 * E + 0.30\left(1 - \frac{|\Delta Score|}{100}\right) + 0.20\left(1 - \frac{|\Delta Attempts|}{4}\right)$$

(Where $E = 1$ if the error type matches, otherwise $E = 0$. Past cases are ranked in descending order, and only successful cases matching the specific error type are eligible for reuse.)

- **Adaptation Logic:** The feedback string and worked example from the retrieved case are copied. Practice volume is dynamically scaled: base practice begins at 3 questions, adds $+\ 2$ questions if the quiz score is below 50, and adds $+\ 2$ questions if previous attempts were $\geq\ 3$.
- **Retention (Learning):** The new problem, adapted solution, and follow-up outcome are assigned the next sequential ID (e.g., C006) and saved atomically to cases.json. Cases with follow-up scores $\geq\ 70$ become immediately eligible for future reuse.

3. Program Execution Screenshots



4. Pair Reflection

- **Paolo C. Gorospe** implemented the case representation schema, built the weighted similarity retrieval function, and wrote the automated test checks.
- **Andreu Liam C. Gamboa** designed the dynamic practice workload adaptation rules, implemented the atomic JSON persistence/retention cycle, and set up the command-line interface.

- We coordinated our tasks by dividing the 4R cycle into data structures/retrieval and adaptation/retention, syncing via VS Code Live Share and reviewing outputs together.
- Our main challenge was preventing practice question inflation across repeated reuses of a case, which we solved by establishing a fixed base budget (3 questions) rather than incrementing the existing count.
- Through this activity, we learned that CBR solves problems efficiently by anchoring decisions to validated past experiences rather than calculating solutions from scratch.
- We also learned that the retention phase transforms static software into an active learning system, as saved cases immediately improve subsequent query matches.

5. Full Source Code

```
"""
TA3 Intelligent Tutoring System using Case-Based Reasoning.
Python 3.10+, standard library only.
"""

import argparse
from copy import deepcopy
import json
import os
from pathlib import Path
import tempfile

ERRORS = {
    "sign": "Incorrect sign when moving a term",
    "distribution": "Multiplier not applied to every term",
    "division": "Only one side divided by the coefficient",
}


def seed_cases():
    """Five fictional successful interventions."""
    examples = [
        (
            "sign",
            40,
            3,
            4,
            80,
            "Use the same inverse operation on both sides.",
            "x + 5 = 12 -> subtract 5 on both sides -> x = 7.",
        ),
        (
            "sign",
            55,
            2,
```

```
            3,
            85,
            "Write the inverse operation before simplifying.",
            "x - 4 = 9 -> add 4 on both sides -> x = 13.",
        ),
        (
            "distribution",
            50,
            3,
            4,
            80,
            "Multiply every term inside the parentheses.",
            "2(x + 3) = 14 -> 2x + 6 = 14 -> x = 4.",
        ),
        (
            "division",
            60,
            2,
            3,
            90,
            "Divide both sides by the coefficient of x.",
            "3x = 18 -> 3x / 3 = 18 / 3 -> x = 6.",
        ),
        (
            "sign",
            75,
            1,
            2,
            95,
            "Solve independently, then substitute to check.",
            "x + 8 = 15 -> x = 7; check: 7 + 8 = 15.",
        ),
    ]
    return [
        {
            "id": f"C{i:03d}",
            "problem": {"error": error, "score": score, "attempts": attempts},
            "solution": {
                "feedback": feedback,
                "example": example,
                "practice_count": count,
            },
            "outcome": {"post_score": post, "status": "successful"},
            "source": "fictional_seed",
        }
        for i, (error, score, attempts, count, post, feedback, example) in enumerate(
            examples, 1
        )
    ]
```

```
def validate_problem(problem):
    if not isinstance(problem, dict) or problem.get("error") not in ERRORS:
        raise ValueError("Choose sign, distribution, or division.")
    for field, low, high in (("score", 0, 100), ("attempts", 1, 5)):
        value = problem.get(field)
        if type(value) is not int or not low <= value <= high:
            raise ValueError(f"{field} must be an integer from {low} to {high}.")


def similarity(problem, past):
    """Weighted similarity in [0, 1]."""
    validate_problem(problem)
    validate_problem(past)
    error_match = float(problem["error"] == past["error"])
    score_match = 1 - abs(problem["score"] - past["score"]) / 100
    attempt_match = 1 - abs(problem["attempts"] - past["attempts"]) / 4
    return 0.50 * error_match + 0.30 * score_match + 0.20 * attempt_match


def retrieve(problem, cases):
    """Rank all cases; reuse only successful cases with the same error."""
    ranked = sorted(
        [(similarity(problem, c["problem"]), c) for c in cases],
        key=lambda item: (item[0], item[1]["id"]),
        reverse=True,
    )
    eligible = [
        (score, c)
        for score, c in ranked
        if c["problem"]["error"] == problem["error"]
        and c["outcome"]["status"] == "successful"
    ]
    if not eligible:
        raise ValueError("No successful case for this error. Ask a tutor.")
    return eligible[0][1], ranked


def adapt(problem, case):
    """Reuse feedback/example; adjust practice to current support needs."""
    validate_problem(problem)
    solution = deepcopy(case["solution"])
    count = 3
    if problem["score"] < 50:
        count += 2
    if problem["attempts"] >= 3:
        count += 2
    solution["practice_count"] = count
    return solution


def validate_cases(cases):
```

```
    if not isinstance(cases, list):
        raise ValueError("Case file must contain a JSON list.")
    seen = set()
    for case in cases:
        if not isinstance(case, dict):
            raise ValueError("Each case must be an object.")
        case_id = case.get("id", "")
        if (
            not isinstance(case_id, str)
            or len(case_id) < 4
            or case_id[0] != "C"
            or not case_id[1:].isascii()
            or not case_id[1:].isdigit()
            or case_id in seen
        ):
            raise ValueError("Case IDs must be unique, e.g. C001.")
        seen.add(case_id)
        validate_problem(case.get("problem"))
        solution = case.get("solution", {})
        if not isinstance(solution, dict):
            raise ValueError("Missing solution object.")
        if any(
            not isinstance(solution.get(k), str) or not solution[k].strip()
            for k in ("feedback", "example")
        ):
            raise ValueError("Each solution needs feedback and an example.")
        count = solution.get("practice_count")
        if type(count) is not int or not 1 <= count <= 20:
            raise ValueError("practice_count must be 1 to 20.")
        outcome = case.get("outcome", {})
        if not isinstance(outcome, dict):
            raise ValueError("Missing outcome object.")
        post = outcome.get("post_score")
        if post is not None and (type(post) is not int or not 0 <= post <= 100):
            raise ValueError("post_score must be 0 to 100, or null.")
        expected = (
            "pending"
            if post is None
            else "successful"
            if post >= 70
            else "needs_review"
        )
        if outcome.get("status") != expected:
            raise ValueError("Outcome status does not match post_score.")


def load_cases(path):
    if not path.exists():
        return seed_cases()
    cases = json.loads(path.read_text(encoding="utf-8"))
    validate_cases(cases)
```

```
    return cases


def save_cases(path, cases):
    """Replace atomically, so an interrupted write does not truncate JSON."""
    validate_cases(cases)
    temp_name = None
    try:
        with tempfile.NamedTemporaryFile(
            mode="w",
            encoding="utf-8",
            dir=path.parent,
            prefix=path.name + ".",
            suffix=".tmp",
            delete=False,
        ) as file:
            temp_name = file.name
            json.dump(cases, file, indent=2)
            file.write("\n")
        os.replace(temp_name, path)
    finally:
        if temp_name and Path(temp_name).exists():
            Path(temp_name).unlink()


def retain(path, cases, problem, solution, post_score, source):
    number = max((int(c["id"][1:]) for c in cases), default=0) + 1
    status = (
        "pending"
        if post_score is None
        else "successful"
        if post_score >= 70
        else "needs_review"
    )
    new_case = {
        "id": f"C{number:03d}",
        "problem": deepcopy(problem),
        "solution": deepcopy(solution),
        "outcome": {"post_score": post_score, "status": status},
        "source": source,
    }
    updated = cases + [new_case]
    save_cases(path, updated)
    return new_case, updated


def display_plan(problem, case, ranked, solution):
    print("\n1. NEW PROBLEM | Linear equations")
    print(f"Error: {ERRORS[problem['error']]}")
    print(f"Quiz score: {problem['score']}/100 | Attempts: {problem['attempts']}")
    print("\n2. RETRIEVAL | Ranked similarity")
```

```
    for score, past in ranked:
        eligible = (
            past["problem"]["error"] == problem["error"]
            and past["outcome"]["status"] == "successful"
        )
        flag = "eligible" if eligible else "excluded"
        print(
            f"{past['id']}: {score * 100:5.1f}% | {past['problem']['error']:<12} | {flag}"
        )
    print(
        f"Retrieved: {case['id']} | Original practice: {case['solution']['practice_count']} questions"
    )
    print("\n3. ADAPTED FINAL SOLUTION")
    print(solution["feedback"])
    print("Example: " + solution["example"])
    print(
        f"Practice: {solution['practice_count']} questions (base 3; +2 if score <50; +2 if attempts >=3)."
    )
    print("Check every answer by substituting x into the original equation.")


def display_cases(cases):
    print(f"\n5. UPDATED CASE BASE | {len(cases)} cases")
    for case in cases:
        p, out = case["problem"], case["outcome"]
        post = "pending" if out["post_score"] is None else out["post_score"]
        print(
            f"{case['id']} | {p['error']:<12} | pre {p['score']:3} | attempts {p['attempts']} | post {str(post):>7} | {out['status']}"
        )


def read_int(prompt, low, high, optional=False):
    while True:
        value = input(prompt).strip()
        if optional and not value:
            return None
        try:
            number = int(value)
            if low <= number <= high:
                return number
        except ValueError:
            pass
        print(f"Enter a whole number from {low} to {high}.")


def run(path, demo=False):
    cases = load_cases(path)
    print("TA3 | CASE-BASED INTELLIGENT TUTORING SYSTEM")
    if demo:
        print("SIMULATED DEMO | Fictional data; temporary case file.")
        problem = {"error": "sign", "score": 35, "attempts": 4}
```

```
    else:
        print("Scope: linear equations; error type is identified by the user.")
        for number, (key, label) in enumerate(ERRORS.items(), 1):
            print(f" {number}. {label} ({key})")
        choice = read_int("Error type (1-3): ", 1, 3)
        problem = {
            "error": list(ERRORS)[choice - 1],
            "score": read_int("Quiz score (0-100): ", 0, 100),
            "attempts": read_int("Attempts (1-5): ", 1, 5),
        }

    case, ranked = retrieve(problem, cases)
    solution = adapt(problem, case)
    display_plan(problem, case, ranked, solution)

    print("\n4. REVIEW AND RETAIN")
    if demo:
        post = 80
        print("Simulated follow-up quiz: 80/100.")
    else:
        print("After practicing, enter a follow-up quiz score.")
        post = read_int(
            "Score (0-100; Enter if not yet tested): ", 0, 100, optional=True
        )

    new_case, updated = retain(
        path,
        cases,
        problem,
        solution,
        post,
        "simulated_demo" if demo else "user_reported",
    )
    print(f"New case saved: {new_case['id']} | {new_case['outcome']['status']}")
    if post is None or post < 70:
        print("Tutor review needed; this case is excluded from future reuse.")

    display_cases(updated)
    reloaded = load_cases(path)
    print(f"\nPersistence verified: {len(reloaded)} cases reloaded from JSON.")

    if demo:
        match, ranks = retrieve(problem, reloaded)
        print(
            f"Same problem after learning: {match['id']} at {ranks[0][0] * 100:.1f}% similarity."
        )


def main():
    parser = argparse.ArgumentParser(description=__doc__)
    parser.add_argument("--demo", action="store_true")
```

```
    parser.add_argument(
        "--cases",
        type=Path,
        default=Path(__file__).with_name("cases.json"),
    )
    args = parser.parse_args()
    try:
        if args.demo:
            with tempfile.TemporaryDirectory() as folder:
                run(Path(folder) / "demo_cases.json", demo=True)
        else:
            run(args.cases.resolve())
    except (ValueError, OSError) as error:
        parser.exit(1, f"Error: {error}\n")
    except (EOFError, KeyboardInterrupt):
        parser.exit(1, "\nCancelled before completion.\n")


if __name__ == "__main__":
    main()
```